Backtracking

Sum of Subsets

[redacted]

Backtracking

- **The state space tree consisting of expanded nodes only is called the *pruned state space tree***
- **The following slide shows the pruned state space tree for the sum of subsets example**
- **There are only 15 nodes in the pruned state space tree**
- **The full state space tree has 31 nodes**

The backtracking method

- A given **problem** has a set of constraints and possibly an objective function
- The **solution** optimizes an objective function, and/or is feasible.
- We can represent the **solution space** for the problem using a **state space tree**
  - The *root* of the tree represents 0 choices,
  - Nodes at depth 1 represent first choice
  - Nodes at depth 2 represent the second choice, etc.
  - In this tree a *path* from a root to a leaf represents a candidate solution

Sum of subsets

- We will assume a binary state space tree.
- The nodes at depth 1 are for including (yes, no) item 1, the nodes at depth 2 are for item 2, etc.
- The left branch includes w_i, and the right branch excludes w_i.
- The nodes contain the sum of the weights included so far

A Depth First Search solution

- Problems can be solved using depth first search of the (implicit) state space tree.
- Each node will save its depth and its (possibly partial) current solution
- DFS can check whether node v is a leaf.
  - If it is a leaf then check if the current solution satisfies the constraints
  - Code can be added to find the optimal solution

A DFS solution

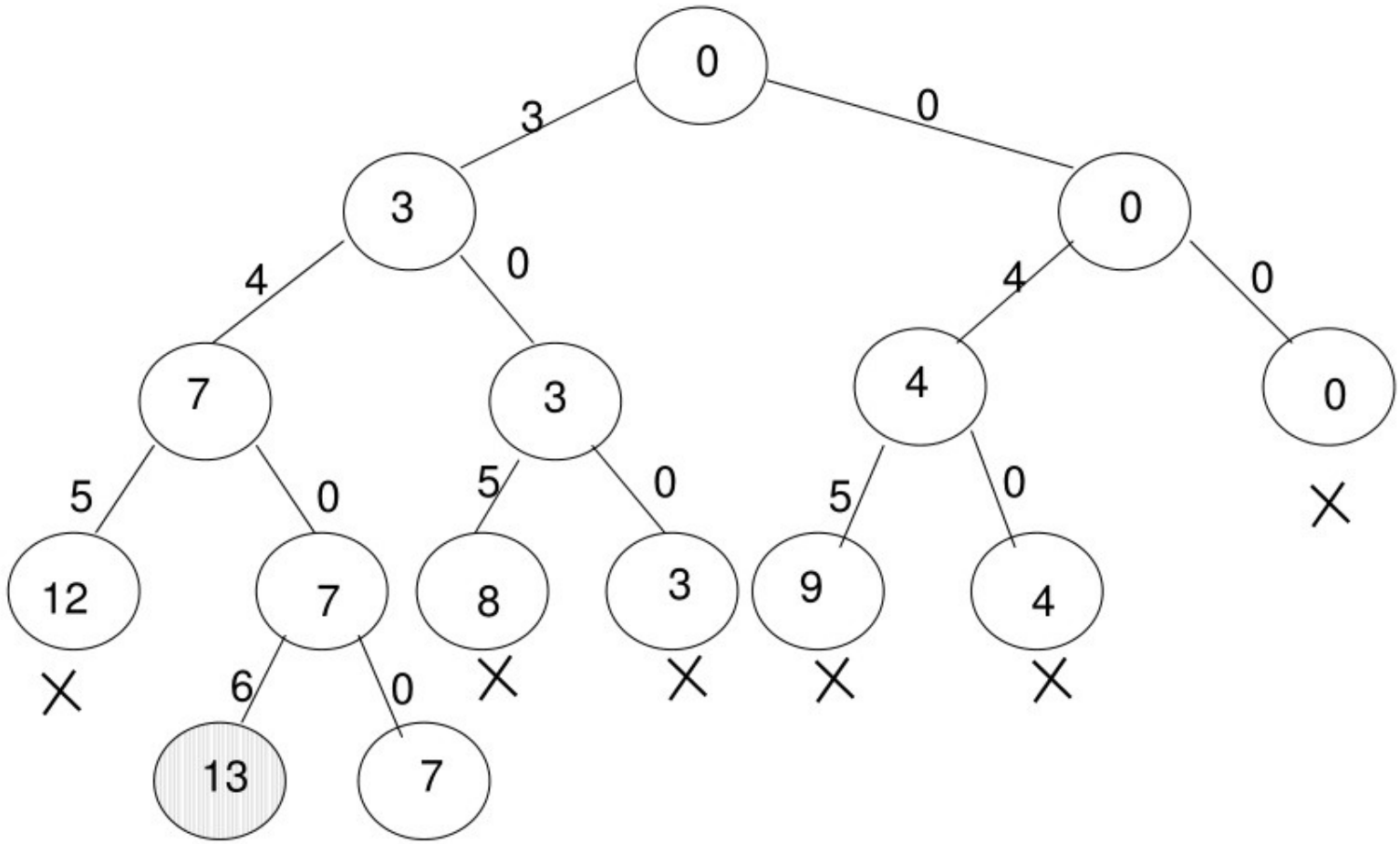
- **Such a DFS algorithm will be very slow.**
- **It does not check for every solution state (node) whether a solution has been reached, or whether a *partial* solution can lead to a *feasible* solution**
- **Is there a more efficient solution?**

Code for sum of subsets

```
dfs(int i, int soFar){
        if (i == n){//found a leaf
                if (soFar == W){
                        print solution inc[1] to inc[n]
                }
                return;
        }//generate space state tree
        inc[i]= 1; //included
        dfs(i+1, soFar+w[i]);
        inc[i]= 0; //excluded
        dfs(i+1, soFar);
    }//initial call dfs(1, 0)
```

A Pruned State Space Tree (find all solutions)

$w_1 = 3,\ w_2 = 4,\ w_3 = 5,\ w_4 = 6;\ \ S = 13$



Sum of subsets problem

Backtracking algorithm

```
void checknode (node v) {
   node u

   if (promising ( v ))
        if (aSolutionAt( v ))
                write the solution
        else //expand the node
                for ( each child u of v )
                        checknode ( u )
```

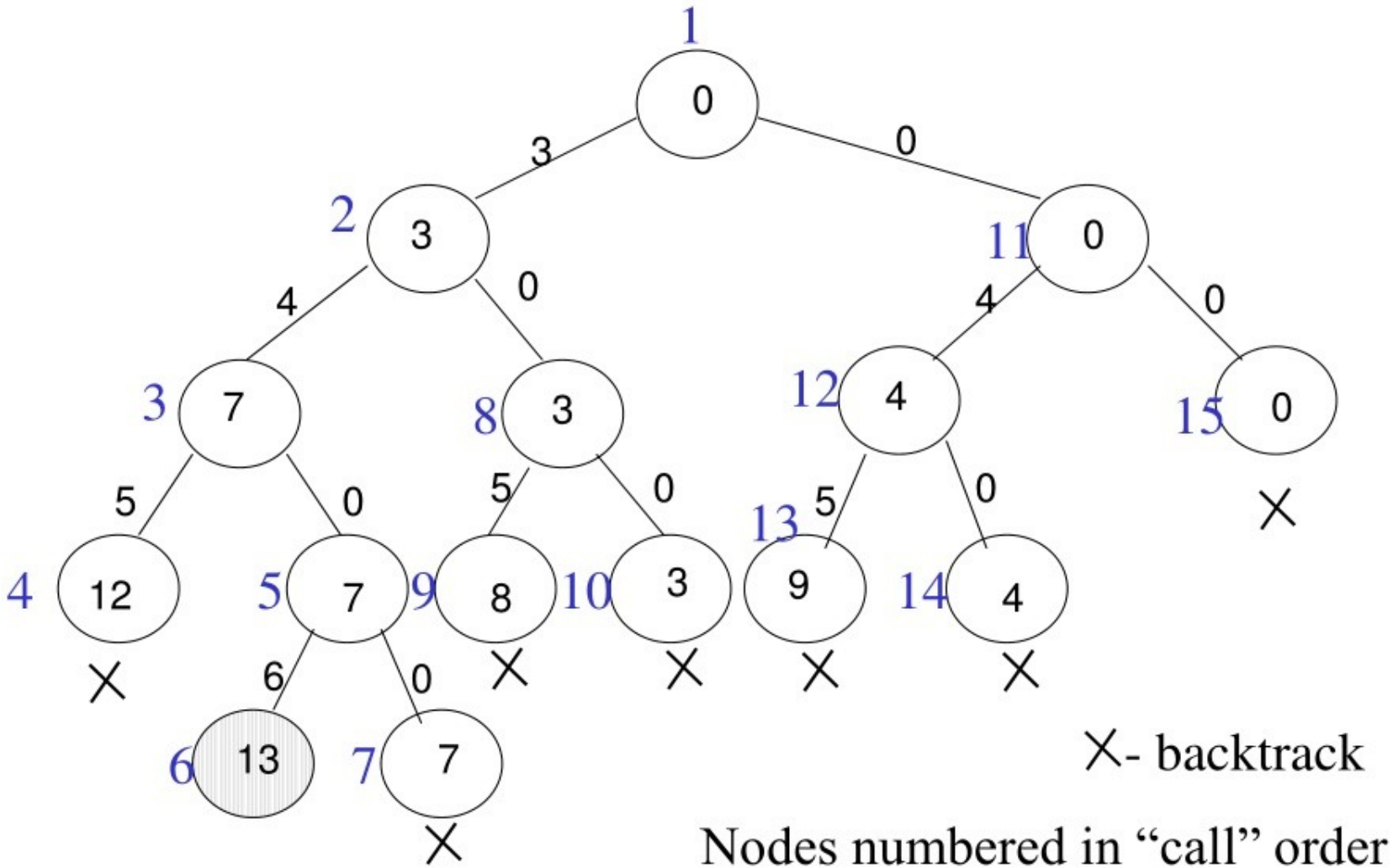
Checknode

- Checknode uses the functions:
  - *promising*(*v*) which checks that the partial solution represented by *v* can lead to the required solution
  - *aSolutionAt*(*v*) which checks whether the partial solution represented by node *v* solves the problem.

Sum of subsets – when is a node "promising"?

- Consider a node at depth i
- *weightSoFar* = weight of node, i.e., sum of numbers included in partial solution node represents

- *totalPossibleLeft* = weight of the remaining items i+1 to n (for a node at depth i)
- A node at depth i is non-promising
  if (*weightSoFar* + *totalPossibleLeft* < S)
  or (*weightSoFar* + w[i+1] > S)
- To be able to use this "promising function" the w_i must be sorted in non-decreasing order

A Pruned State Space Tree

$w_1 = 3,\ w_2 = 4,\ w_3 = 5,\ w_4 = 6;\ \ S = 13$